

ATANDA Command Centre — Ingestion Product Design Document (IPDD)

Version: 1.1 — Investor Preview (IPDD rename)**Status:** Pre-launch, code-complete**Document type:** IPDD (human-authored INPUT — see Terminology)**Last updated:** 20 May 2026

1. Vision

ATANDA Command Centre is a subscription portal that puts a deterministic prompt-engineering pipeline — **FORGE.BONSAI HARNESS** — in the hands of solo founders, applied AI engineers, and product teams. A user logs in, runs a single coherent session, and the HARNESS walks their raw idea through seven atomic engines (F1 !' F7, with an F6-VDJ branch and an optional DE-SPC synthesiser) until it emerges as a **SPARTAN-certified MVP-PDD** with a public verification URL.

The HARNESS itself is a **PDD blueprint application** — an instruction layer of ordered atomic prompts that drives Claude through a deterministic pipeline. It is **not** an SPC; the artefacts the HARNESS *produces on the user's behalf* (Atomic Prompts, CELL Micro Agent Birth Packages, Micro PDDs, full SPCs, 4-Part ATLAS PDDs, MVP PDDs) are the SPCs/PDDs.

2. Audience & monetisation

Tier	Price	Who	What unlocks
Explorer	Free	First-time visitors	F1 – F4, exemplar library, 3 sessions, daily rate-limited
Practitioner	Monthly / Yearly	Working builders	F5 + F6 + F7 + ATLAS PDD + SPARTAN MVP-PDD certification + higher daily caps
Architect	Monthly / Yearly	Power users / agencies	DE-SPC synthesiser, AISE badge, priority caps, exports (CSV/PDF)
Ingestion credit	**\$199.99 / project** (one-time)	Anyone signed in (Explorer included)	One IPDD !' one PWDD session, decoupled from monthly tier

All paid flows route through **Stripe Checkout**; the Ingestion credit is a [mode=payment](#) one-time purchase that drops a row into a [ingestion_credits](#) ledger and is **atomically claimed** at the start of an ingest request.

3. Terminology — IPDD vs PWDD (canonical)

The bare term "PDD" was overloaded: it meant both the human-authored INPUT to the INGESTION

ENGINE **and** appeared as a suffix on the HARNESS-produced OUTPUTS (Micro PDD, ATLAS PDD, MVP PDD). To remove the ambiguity, the **INPUT** is now formally named **IPDD**. The qualified OUTPUT names are unchanged.

- **IPDD = INPUT.** An *Ingestion Product Design Document* — historically just "PDD". A **human-authored, pre-ingestion** artefact a user already owns (a Product Design Document, Software Design Document, concept note, spec sheet, brief, etc.). It is what the INGESTION ENGINE consumes — never what the HARNESS produces.
- **PWDD = OUTPUT.** A *PromptWare Design Document* is the **HARNESS-certified, post-ingestion** outcome of a session seeded by ingesting an IPDD. Sessions with `origin='ingested'` produce PWDDs; manual sessions (`origin='manual'`) produce regular MVP-PDDs.
- **Qualified output PDDs are untouched.** *Micro PDD* (F4), *ATLAS PDD* (F6), and *MVP PDD* (F7) keep their names — they are always prefix-qualified, so they can never collide with the INPUT term.
- **Internal enum stays.** The `sourceDockKind` Postgres / API enum value `product_design_document` is unchanged (no migration, no codegen drift); only the user-facing label moves to "Ingestion Product Design Document (IPDD)".
- The UI must never call an INPUT a "PWDD" and never call a HARNESS OUTPUT an "IPDD".

This very document is itself an **IPDD** — a human-authored snapshot of the live codebase as an INPUT artefact.

4. System architecture

4.1 Monorepo layout (pnpm workspaces)

```
artifacts/  
% % % api-server/           Express 5 API, Clerk + Stripe + Drizzle + Pino  
% % % command-centre/       React 19 + Vite SPA – the portal shell  
% % % mockup-sandbox/       Vite component preview server (design surface)  
lib/  
% % % api-spec/             OpenAPI 3 spec – single source of truth  
% % % api-zod/              Generated Zod validators (per operationId)  
% % % api-client-react/     Generated React Query hooks  
% % % db/                   Drizzle schema + migrations  
% % % email/                Resend wrapper, dry-run console fallback  
% % % export/               CSV + PDF (PDFKit) exporters  
% % % integrations-anthropic-ai/ Claude Sonnet 4 wrapper via Replit AI proxy
```

4.2 Stack

- **Runtime:** Node.js 24, TypeScript 5.9, pnpm workspaces
- **API:** Express 5 + Clerk (Replit-managed) + Stripe + Pino structured logging
- **DB:** PostgreSQL + Drizzle ORM; **app-layer authorization, no Postgres RLS** — every Drizzle query that touches user-owned data filters on `req.localUser.id`
- **Validation:** Zod (`zod/v4`) + `drizzle-zod`; request bodies validated by **generated** Zod (named by `operationId`, e.g. `CreateSessionBody`)
- **Codegen:** Orval reads `openapi.yaml` !' emits Zod schemas + React Query hooks; response shapes returned as plain TS-typed objects
- **Build:** esbuild !' CJS bundle for the API
- **LLM:** Claude Sonnet 4 (`claude-sonnet-4-6`) via Replit AI Integrations proxy (`AI_INTEGRATIONS_ANTHROPIC_*` env, no user-supplied key)
- **Observability:** Sentry (gated on `SENTRY_DSN` / `VITE_SENTRY_DSN`, no-op when unset), per-engine telemetry rows in `harness_engine_runs`

4.3 Contract-first discipline

Every endpoint is **declared in `openapi.yaml` first**:

- The server validates inputs with the generated Zod (`CreateSessionBody.safeParse(req.body)`)
- The client calls generated React Query hooks (`useCreateSession`, `useGetIngestionCredits`, ...)
- Response shapes have no generated Zod — returned as plain objects typed by the generated TS interface
- Drift is caught by `pnpm run typecheck` (libs + leaves)
- **Never** name a component `<OperationIdPascal>Body` or use inline request bodies (causes TS2308)

5. Domain model

5.1 Database schema (`lib/db/src/schema/\*.ts`)

Table	Purpose
`users`	Local user mirror of Clerk identity; `role` (user / admin), JIT-synced
`subscribers`	Stripe subscription state + per-engine daily counters `f{1..7}_today`
`harness_sessions`	One row per FORGE.BONSAI run; `origin` " {`manual`, `ingested`} ; `ingestionId` FK
`harness_artifacts`	Each engine's output (Atomic Prompt, MA, Micro PDD, SPC, ATLAS PDD, MVP PDD); `spcOrigin` distinguishes DE-SPC
`harness_engine_runs`	Per-call telemetry (best-effort, never breaks a request)
`harness_escalations`	Tracks F3 !' F5 escalation paths; bypasses tier gate for the affected session

<code>`harness_feature_state`</code>	Per-session unlock/lock state of each engine
<code>`ingestion_documents`</code>	Normalised payload from <code>`pdf-parse`</code> / <code>`mammoth`</code> + Claude — <code>`{detectedTitle, sourceDocKind, summary, seedPrompt}`</code>
<code>`ingestion_credits`</code>	One-time-purchase ledger: <code>`(status='available'</code>
<code>`stripe_webhook_events`</code>	PK on <code>`event_id`</code> — idempotency net for Stripe retries
<code>`command_centre_badges`</code>	Persisted AISE badge (URL-verified)
<code>`context_craft_badges`</code>	Persistent pillar badges (S/R/I/D/F/E/C), unique on <code>`(user_id, pillar)`</code> , upserted via <code>`GREATEST(bestScore, new)`</code>
<code>`pricing_content`</code>	Admin-editable pricing copy

5.2 Auth & authorization

- **Clerk** (Replit-managed) at the edge: `clerkProxyMiddleware` mounts `CLERK_PROXY_PATH` before any body parsers
- **JIT mirror** (`ensureLocalUser` in `auth.ts`): strict — requires a successful `clerkClient.users.getUser` before inserting a local row. Clerk 404 != 401 `ClerkIdentityNotFoundError`. Existing local users short-circuit before the Clerk call so a Clerk outage **never** locks out provisioned accounts.
- **Admin role** = Clerk `publicMetadata.role === "admin"` OR email in the `ADMIN_EMAILS` env allowlist, mirrored to `users.role` at JIT-sync time.
- **Tier gating + escalation bypass** (`tier.ts`): `requireTier("PRACTITIONER")` on F5 / F6 / F7; if the request body's `sessionId` has a `harness_escalations` row, the gate is bypassed for that session.
- **Account delete is external-first, local-last** (`POST /api/me/delete`): cancel Stripe sub (hard-fail 502 if reachable & a subscription id exists; benign on 404) != `clerkClient.users.deleteUser` (hard-fail 502 on error) != only then cascade-delete the local `users` row != best-effort goodbye email. Re-ordering would orphan Stripe subs and let stale tokens JIT-recreate shell users.

6. The HARNESS — engines & flow

6.1 Per-session pipeline

```

F1 diagnose raw prompt
  !"
F2 build Atomic Prompt
  !"
F3 grow CELL Micro Agent (MA) Birth Package          !• may escalate to F5
  !"
F4 convert to Micro PDD
  !"
F5 build full SPC                                     [PRACTITIONER+]
  !"
F6 draft 4-Part ATLAS PDD  % % %, % % >  F6-VDJ recommendation
  !"                                     %
F7 compress to SPARTAN-certified MVP PDD              [PRACTITIONER+]
  + public verification URL

```

Optional branch: **DE-SPC** ([engineId=8](#), [POST /api/harness/evolve](#)) — gated by [requireAuth](#) + [requireTier\("PRACTITIONER"\)](#) + [requireAspeBadge](#). Persists via [persistArtifact](#) then [UPDATE](#) to set [spcOrigin='digitally_evolved'](#).

6.2 Engine code (`artifacts/api-server/src/engines/`)

- [prompts.ts](#) — single source of truth for all system prompts, v2 with anchored rubrics, calibrated escalation thresholds, deterministic FORGE script, audit-grade SPARTAN math
- [shared.ts](#) — [callClaude](#) / [callClaudeJson](#) with optional [RunContext](#) that writes per-call telemetry to [harness_engine_runs](#)
- [f1.ts](#) – [f7.ts](#), [f6vdj.ts](#), [de.ts](#) — one engine per file, each returning the persistable artefact shape

6.3 Rate limiting

- Daily counters live on the **subscriber row** ([f{1..7}_today](#))
- Guarded [POST /api/cron/reset-harness-limits](#) (header [x-cron-secret](#)) resets them
- Per-tier caps enforced inline before each Claude call

7. INGESTION engine — per-project billing model

The Ingestion Engine is **fully decoupled from monthly subscription tiers**:

- Purchase** — `POST /api/billing/ingestion/checkout` opens a Stripe Checkout in `mode=payment` for `STRIPE_PRICE_INGESTION_PROJECT`, stamped with `metadata.kind=ingestion_credit`. Reuses the existing JIT Stripe-customer pattern.
2. **Webhook** — `checkout.session.completed` forks on `mode=payment && metadata.kind=ingestion_credit` !' inserts an `ingestion_credits` row. **Idempotent on** ``stripe_checkout_session_id` UNIQUE` (second net beyond the outer `stripe_webhook_events` PK).
 3. **Claim** — `POST /api/ingest` atomically claims one available credit *before* spending any LLM tokens, via raw SQL: ```sql UPDATE ingestion_credits SET status='consumed', consumed_at=now() WHERE id = (SELECT id FROM ingestion_credits WHERE user_id=$1 AND status='available' ORDER BY purchased_at ASC FOR UPDATE SKIP LOCKED LIMIT 1) RETURNING id; ``` Concurrent ingests **cannot** double-spend a single credit.
 4. **Refund-on-failure** — a `success` flag plus `finally` block releases the claimed credit on **every** non-success path: invalid input, too-short text, normalisation failure, thrown exception. The user is only billed when an `ingestion_documents` row is persisted.
 5. **Link** — on success the credit is permanently linked to `ingestion_documents.id` via `linkCreditToDocument` (best-effort; failure leaves the credit consumed-but-unlinked, with a warning log).
 6. **Session start** — `POST /api/ingest/:id/start-session` creates a HARNESS session with `origin='ingested'` and `ingestionId` set, F1 unlocked, others LOCKED. **Not** charged again — the credit pays for the full document !' PWDD session.
 7. **Balance** — `GET /api/ingestion-credits` returns `{available, consumed, total}`; the `/ingest` page disables the NORMALISE button until at least one credit exists.

8. Stripe integration

- **Raw-body webhook mount order** in `app.ts` is load-bearing: `CLERK_PROXY_PATH !/api/webhooks/stripe (express.raw({type:"application/json"})) !' cors !' json/ urlencoded !clerkMiddleware !/api routes`
- **Idempotency** — every event id is recorded in `stripe_webhook_events` (PK) on receipt; replays return `{ok:true, replay:true}` without re-executing handlers. If a handler throws, the idempotency row is rolled back so Stripe can legitimately retry.
- **Unknown price ids fail loudly** — `applySubscription` throws `UnknownPriceError` !' HTTP 400, logged with `{priceId, customerId}`. Always add the new `STRIPE_PRICE_*` env before launching a new tier.

9. Front-end (`artifacts/command-centre`)

- React 19 + Vite + Wouter routing + TanStack Query + Tailwind + shadcn/ui
- Pages: `landing`, `pricing`, `sign-in/up` (Clerk), `command` (session shell), `session-new`, `session-detail`, `sessions`, `ingest`, `exemplars`, `prompts`, `quests`, `verify`, `billing`, `account`, `demo`, `not-found`
- F1–F7 workspaces fully built inside `session-detail`
- **Exemplar library** with a "Fork to session" CTA
- `/quests` page renders triangle badges for the seven Context Craft pillars and the AISE quest badge
- ``BILLING_ENABLED` preview flag` (`src/lib/billing-flag.ts`) — when `false`, both Stripe-Checkout CTAs fall back to a `mailto:` access-request flow, gated by `ACCESS_REQUEST_EMAIL`
- `/pricing` card structure: three tier cards (Explorer / Practitioner / Architect) + a standalone full-width **INGESTION ENGINE — PER PROJECT** card at **\$199.99 / project** with "Works on any account tier (Explorer included)" + "Credits never expire"

/ingest page: credit-balance card at top (**N CREDITS AVAILABLE** or **NO CREDITS YET** + BUY CTA), NORMALISE disabled until balance > 0, balance refetches after every successful ingest

10. Quests & badges

Two badge surfaces, deliberately different lifecycles:

- **AISE badge** — single, persisted to [command_centre_badges](#) after URL verification. The verifier ([lib/badges.ts#head0k](#)) **refuses RFC1918 / loopback / link-local / metadata** (**169.254.169.254**) / **CGNAT / multicast** addresses and rejects anything not [https](#):. Do not relax without a deliberate SSRF review.
- **Quest badges (live)** — re-counted on every [/me/badges](#) call from current SPC / MA / PDD counts, never stored
- **Context Craft pillar badges (S / R / I / D / F / E / C)** — persistent, **auto-awarded** from inside [persistArtifact](#) whenever an artefact carries a [jcse](#) pillar breakdown (F1, F2). Threshold = pillar sub-score "e 6. Rows live in [nontext_craft_badges](#) (unique on ([user_id](#), [pillar](#))), upserted via [GREATEST\(bestScore, new\)](#). Fire-and-forget — failures **never** break an engine response.

11. API surface (selected operations)

Operation	Method + path	Notes
<code>`healthCheck` / `healthDeep`</code>	GET <code>`/api/healthz`</code> , <code>`/api/health/deep`</code>	Liveness + dependency probes
<code>`getMe`</code> , <code>`updateMyProfile`</code> , <code>`deleteMyAccount`</code>	<code>`/api/me`</code>	External-first delete
<code>`createSession`</code> , <code>`listSessions`</code> , <code>`getSession`</code> , <code>`updateSession`</code> , <code>`deleteSession`</code>	<code>`/api/sessions`</code>	Owner-scoped
<code>`harnessF1` ... `harnessF7Stream`</code>	<code>`/api/harness/f`</code>	SSE streaming on F3 + F7
<code>`harnessF6Vdj`</code> , <code>`harnessEscalationsStream`</code> , <code>`harnessEvolve`</code>	<code>`/api/harness/`</code>	VIBE DJ, escalation, DE-SPC
<code>`billingCheckout`</code> , <code>`billingPortal`</code>	<code>`/api/billing/`</code>	Subscription Stripe Checkout
<code>**`billingIngestionCheckout`**</code>	POST <code>`/api/billing/ingestion/checkout`</code>	One-time \$199.99 credit
<code>**`getIngestionCredits`**</code>	GET <code>`/api/ingestion-credits`</code>	<code>`{available, consumed, total}`</code>
<code>`getSessionIngestion`</code> , <code>`getIngestion`</code> , <code>`startSessionFromIngestion`</code>	<code>`/api/ingest`</code>	Document ! PWDD pipeline
<code>`stripeWebhook`</code>	POST <code>`/api/webhooks/stripe`</code>	Raw body, idempotent
<code>`verifyCertificate`</code>	GET <code>`/api/verify/:certId`</code>	Public — backs MVP-PDD URL

<code>`exportMyData`, `exportMySpcsCsv`, `exportMySpcsPdf`</code>	<code>`/api/me/export*`</code>	CSV + PDF
<code>`cronResetHarnessLimits`</code>	<code>POST `/api/cron/reset-harness-limits`</code>	<code>`x-cron-secret`</code> header
<code>`listExemplars`, `getExemplar`</code>	<code>`/api/exemplars*`</code>	Public catalogue
<code>`getPricing`, `putPricing`</code>	<code>`/api/pricing`</code>	Admin-editable copy
<code>`claimAiseBadge`, `listMyBadges`, `listMyContextCraftBadges`</code>	<code>`/api/me/*badges*`</code>	Quest surface

12. Environment

Required

- `DATABASE_URL` (auto-provisioned)
- `CLERK_SECRET_KEY`, `CLERK_PUBLISHABLE_KEY`, `VITE_CLERK_PUBLISHABLE_KEY` (auto-provisioned)

Optional (feature-gated)

- `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`
- `STRIPE_PRICE_{PRACTITIONER, ARCHITECT}_{MONTHLY, YEARLY}`
- `STRIPE_PRICE_INGESTION_PROJECT` — **set this to a \$199.99 one-time price id before go-live**
- `ANTHROPIC_API_KEY` (or use the Replit AI Integrations proxy — preferred)
- `ADMIN_EMAILS` (comma-separated)
- `CRON_SECRET`
- `RESEND_API_KEY` + `EMAIL_FROM` (falls back to `[email:dry-run]` console log when unset)
- `SENTRY_DSN` + `VITE_SENTRY_DSN` (no-op in dev when unset)
- `PUBLIC_BASE_URL` — used to build cert verify URLs in outgoing emails

13. Operational gotchas (load-bearing)

1. **OpenAPI naming.** Never name a component `<OperationIdPascal>Body` or use inline request bodies (TS2308 collision in generated `api.ts` vs `types/`).
2. **Generated Zod is operationId-shaped.** `CreateSessionBody.safeParse(req.body)`, not `SessionInput`.
3. **Clerk proxy mounts before body parsers.** Order in `app.ts` matters.
4. **Never call service ports directly.** Always `localhost:80` (e.g. `curl localhost:80/api/healthz`), never `localhost:5000`.
5. **Stripe webhook is idempotent + raw-body.** Replays are no-ops; thrown handlers roll back the idempotency row so Stripe can legitimately retry.
6. **Unknown Stripe price ids fail loudly.** Add new `STRIPE_PRICE_*` env before launching a tier.
7. **Per-engine telemetry is best-effort.** Failures log a warning, never break the request.
8. **AISE URL verifier blocks private IPs** (RFC1918 / loopback / link-local / metadata / CGNAT / multicast) and non-https. Do not relax without an SSRF review.
9. **PDFKit needs `@swc/helpers` as a real dep** of `@workspace/api-server` — esbuild does not pull it in transitively.
10. **DE-SPC** requires `requireAuth` + `requireTier("PRACTITIONER")` +

`requireAspeBadge; persistArtifact` then `UPDATE` to set `spcOrigin='digitally_evolved'`.

11. **Quest badges are computed live.** Only AISE persists.
12. **Context Craft awarder is fire-and-forget.** Never block an engine response.
13. **Account delete is external-first, local-last.** Re-ordering orphans Stripe subs.
14. **`ensureLocalUser` is strict on first JIT sync.** Existing users short-circuit before Clerk — outages don't lock out provisioned accounts.
15. **Sentry init must run before any other module-side-effect imports** in `app.ts`.
`@opentelemetry/*` is NOT externalised in the esbuild bundle (Sentry needs it bundled into the CJS output).
16. **Ingestion credit refund logic.** `try { ... success=true; ... } catch { ... } finally { if (!success) await releaseIngestionCredit(creditId) }` — fixes early-return failure paths that would otherwise burn a paid credit.

14. Build, test, deploy

```
pnpm run typecheck           # full typecheck across all
pnpm run build                # typecheck + esbuild bundles
pnpm --filter @workspace/api-spec run codegen  # regenerate Zod + React
pnpm --filter @workspace/db run push          # push schema (dev only)
pnpm --filter @workspace/api-server run dev   # workflow: artifacts/api-
pnpm --filter @workspace/command-centre run dev # workflow: artifacts/
command-centre
```

Three workflows are pre-wired in the Replit environment:

- `artifacts/api-server`: `API Server`
- `artifacts/command-centre`: `web`
- `artifacts/mockup-sandbox`: `Component Preview Server`

Investor-preview publishing keeps `BILLING_ENABLED=false` so all Stripe CTAs fall back to a mailto access-request flow — perfect for showing the surface without live charges. **Go-live publishing** flips `BILLING_ENABLED=true` and requires every `STRIPE_PRICE_*` env (including `STRIPE_PRICE_INGESTION_PROJECT`) to be set.

15. What "done" looks like (current state)

- ' Portal shell + landing + pricing + auth (Clerk) + account + billing pages
- ' All seven HARNESS engines (F1–F7) + F6-VDJ + DE-SPC, with v2 prompts
- ' Per-session escalation bypass, per-engine telemetry, daily rate limits
- ' Stripe subscription tiers (Practitioner / Architect, monthly + yearly) with idempotent webhook
- ' **Standalone per-project Ingestion Engine at \$199.99, atomic claim, refund-on-failure, idempotent webhook** (*this milestone*)
- ' Exemplar library + "Fork to session"
- ' Quests surface (live quest badges + persistent Context Craft pillar badges + AISE badge with

SSRF-safe verifier)

- ' CSV + PDF exporters, public cert verification URL
- ' Account delete (external-first, local-last)
- ' Sentry hooks (DSN-gated), Pino structured logging, health/deep probes
- ' Contract-first OpenAPI with generated Zod + React Query — 0 hand-written hooks

16. What ships next (post-launch backlog)

- Repair job for [consumed-but-unlinked](#) ingestion credits (rare race window between document insert and link)
 - Generated Zod parse on [billingIngestionCheckout](#) body (currently accepts empty object — low-risk but inconsistent with the rest of the routes)
 - Stripe Tax / address collection on the one-time ingestion checkout
 - Admin dashboard for credit ledger inspection
 - Webhook event archival / cold-storage rotation for [stripe_webhook_events](#)
-

Authored as the canonical INPUT IPDD for the ATANDA Command Centre codebase, intended both as investor-facing documentation and as a seed for a future HARNESS ingestion run.